Option Pricing - the Black Scholes Model

The Black Scholes formula is one of the most popular financial instruments in use. It was derived by Fisher Black, Myron Scholes, and Robert Merton in 1973, and since then it has become the primary tool for derivative pricing.

The original framework that was developed by the three scientists considered the pricing of a European call or put option and assumed that there were efficient markets, an absence of transaction costs, no dividend payments, and a known volatility and risk-free rate. Some of these hypotheses can be used to calculate an option price in practice. In this chapter, we are going to understand derivatives and look at the Black Scholes formula for option pricing and Euler Discretization.

In this chapter, we will be focusing on the following topics:

- An introduction to derivative contracts
- Using the Black Scholes formula for option pricing with Python
- Using Monte Carlo in conjunction with Black Scholes
- Using Monte Carlo in conjunction with Euler Discretization in Python

Technical requirements

In this chapter, we will be using the Jupyter Notebook for coding purposes. We will also be using the pandas, NumPy, matplotlib, and SciPy libraries.

An introduction to the derivative contracts

A derivative is a financial instrument whose price is determined, or derived, based on the development of one or more underlying assets, such as stocks, bonds, interest rate commodities, and exchange rates. It is a contract involving at least two parties and describes how and when the two parties will exchange payments. Some derivative contracts are traded in regulated markets, while others that are traded over the counter are not regulated.

Derivatives traded in regulated markets have a uniform contractual structure and are much simpler to understand. Originally, derivatives were used as a hedging instrument. Companies interested in buying these contracts were mostly concerned about protecting their investments.

However, with time, a great deal of innovation was introduced to the scene. So-called financial engineering was applied and new types of derivatives appeared. Nowadays, there are many types of derivatives. Some are rather complicated and difficult to understand, even for those with a lot of experience in finance.

There are three groups of people who tend to be interested in dealing with derivatives:

- People interested in hedging their investments
- Speculators
- Arbitrageurs

In this chapter, we will learn about the four main types of financial derivatives and how they function:

- Forward
- Future
- Option

  Swap

Forward contracts

A forward contract is a tweaked contract between two gatherings, where settlement happens on a particular date in the future at a cost incurred today. Forward contracts are not traded in the standard stock exchange and, as a result, they are not standardized, making them particularly useful for hedging. The primary characteristics of forward contracts are as follows:

- They are reciprocal contracts and are consequently presented to the opposite party
- Each agreement is hand-crafted and is therefore unique in regard to the measure of the contract, the lapse date, the asset type, and the asset quality
- The agreement must be settled by conveyance of the benefit on the lapse date

Future contracts

A future contact is a contract that is used to buy or sell underlying assets, such as stocks, bonds, or commodities at a specified time. A future contract is a lawful consent to purchase or offer a specific commodity or asset at a cost at a predetermined time later on. Future contracts are similar to forward contracts, but they are standardized and regulated so that they may be traded in the future. They are often used to speculate on commodities.

The purchaser of a future contract assumes the commitment of purchasing the underlying assets when the future contract terminates. The seller of the future contract assumes the commitment of providing the underlying asset on the expiry date. Every future contract has the following features:

- A purchaser
- A vendor
- A cost
- An expiry date

Option contracts

An option contract is a contract that gives someone the right, but not the obligation, to buy (call) or sell (put) security or an other financial asset. A call option gives the purchaser the privilege of purchasing the asset at a given cost, called the **strike price**. While the holder of the call option has the privilege of requesting an offer from the seller, the vendor or the seller has the right to sell but not necessarily the commitment to do so. If a purchaser wants to purchase the underlying asset, the merchant needs to offer it, but they don't have the obligation to do so.

Similarly, a put option gives the purchaser the privilege of selling the asset at the strike price to the purchaser. Here, the purchaser has the privilege to offer, and the seller has the commitment to purchase. In every option contract, the privilege to exercise the option is vested with the purchaser of the agreement. The seller of the agreement has the right to sell, but not the commitment to do so. As the seller of the agreement bears the commitment, they are paid a cost, called a premium.

Swap contracts

A swap contract is used for the exchange of one cash flow for another set of future cash flows. A swap refers to the exchange of one security for another, based on different factors.

The main advantages of swap contracts are listed here:

- **Converting financial exposure**: Swap contracts can be used to convert currencies to obtain debt financing at a reduced interest rate. This is brought about by the comparative advantages that each counter-party has in their national capital market, and/or the benefits of hedging long-run exchange rate exposure.
- **Comparative advantages**: The different types of debt instruments show that there exists an arbitrage opportunity due to mispricing of the default risk premiums.
- **Speculations**: Swap contracts allow us to speculate on interest rates, currencies, and so on.

Using the Black Scholes formula for option pricing

The Black Scholes formula calculates the value of an option. An option is the freedom to choose whether to acquire a stock. The holder of the option may decide if they want to buy the stock, but they may also decide that they are better off without doing so. This freedom is valuable to every investor, so it has a price.

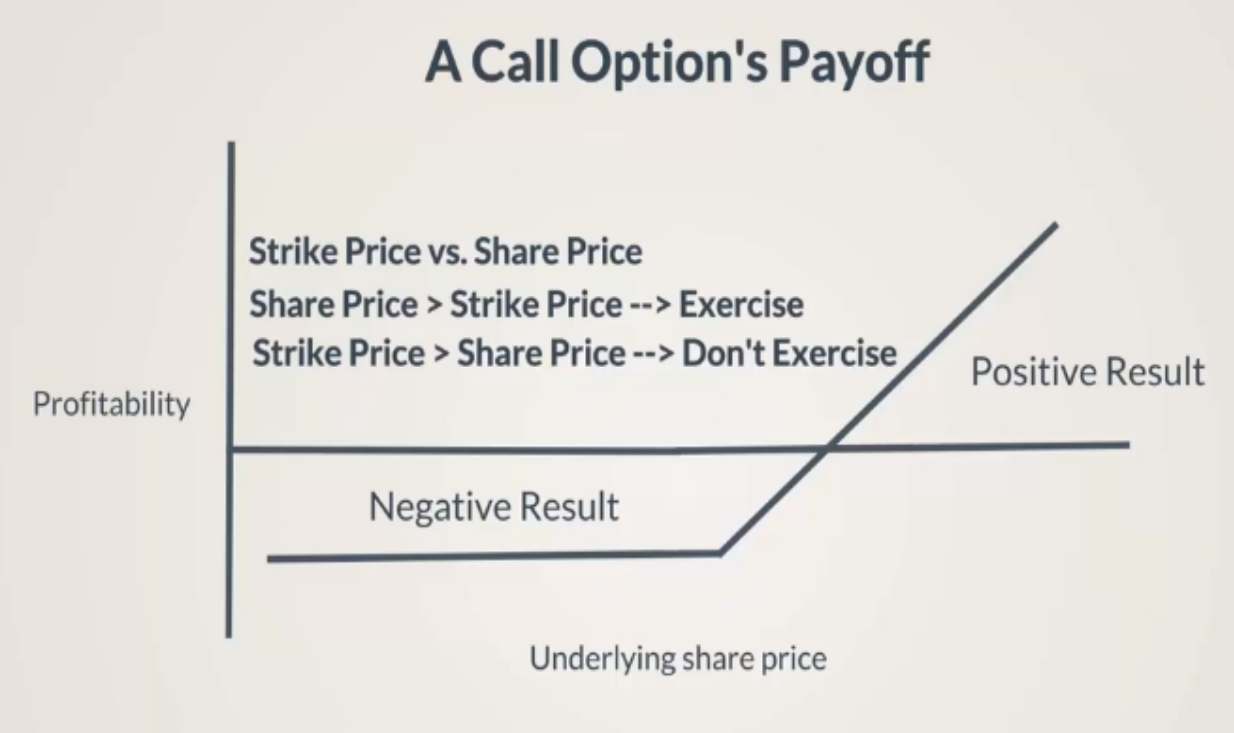
Here is a diagram representing the payoff of a call option:

The *y* axis shows the profitability of an investor who buys the call option, while the *x* axis shows the development of the underlying share price for which the investor has an option. When the option expires, the owner will compare the strike price and the actual market price of the underlying share.

The strike price is the price at which the derivative can be bought or exercised. This term is most commonly used to describe the index and the stock options. For call options, the strike price is the price at which the stocks or the derivatives can be bought by the buyer until the expiration date. For put options, the strike price is the price at which shares can be sold by the option buyer.

If the strike price is lower than the market price, the owner of the option will exercise it. Conversely, if the strike price is higher than the market price, they won't exercise the option, because they must buy the share at a higher price than its market price.

The profitability of the investor therefore looks as follows:



At first, the investor buys the option. They pay the money and so their profitability is negative. Then, when the expiration date comes around, they are able to use this option if the price of the underlying share is higher than the strike price. Even if the market price is higher than the strike price, this doesn't mean the investor will profit from the deal. They need a price that is significantly higher to reach a break-even point and profit from the deal.

The Black Scholes formula provides an intuitive way to calculate the option prices. The formula is as given here:

$$C(S,t) = N(d_1)S - N(d_2)Ke^{-r(T-t)}$$
$$d_1 = \frac{1}{s\sqrt{(T-t)}}[ln(\frac{S}{K}) + (r + \frac{S^2}{2})(T-t)]$$
$$d_2 = d_1 - s\sqrt{T-t}$$

Here, we have the following:

- *S* = the current stock price
- *K* = the option strike price
- *T* = the time when the option was exercised
- *t* = the time until the option expires
- *r* = the risk-free interest rate
- *s* = the sample standard deviation
- *N* = the standard normal distribution

- *e* = the exponential term
- *C* = the call premium

The first component, d_1, shows us how much we are going to get if the option is exercised. The second component, d_2, is the amount we must pay when exercising the option. If we have all the inputs necessary to calculate d_1 and d_2, we shouldn't have a problem in obtaining these values and applying them in the preceding formulas.

This is the key to understanding the formulas. In the first component, to find d_1, we are multiplying the probability of exercising the option by the amount received when the option is exercised. In the second component, to find d_2, we subtract the value of the amount we must pay to exercise the option. All this is done if the development of the stock's share price follows a normal distribution.

In short, the Black Scholes formula calculates the value of a call by taking the difference between the amount you get if you exercise the option, minus the amount you have to pay if you exercise the option.

Calculating the price of an option using Black Scholes

Our goal in this section will be to calculate the price of a call option using Python. We will apply the Black Scholes formula that we introduced in the previous section.

Let's begin by importing the necessary libraries, as shown in the following code:

```
In[1]:
import numpy as np
import pandas as pd
from pandas_datareader import data as wb
from scipy.stats import norm
```

Now, we will create two functions that will allow us to calculate d_1 and d_2, which we need to apply the Black Scholes formula and price a call option. The parameters to include are the current stock (*S*), the strike price (*K*), the risk-free interest rate (*r*), the standard deviation (*stdev*), and the time (*t*) measured in years.

The following are the equations of the two functions, d_1 and d_2:

$$d_1 = \frac{ln(\frac{S}{K}) + (r + \frac{stdev^2}{2})t}{s.\sqrt{t}}$$

$$d_2 = d_1 - s.\sqrt{t} = \frac{ln(\frac{S}{K}) + (r - \frac{stdev^2}{2})t}{s.\sqrt{t}}$$

The code is as follows:

```
In[2]:
def d1(S, K, r, stdev, T):
    return (np.log(S / K) + (r + stdev ** 2 / 2) * T) / (stdev * np.sqrt(T))
```

```
def d2(S, K, r, stdev, T):
    return (np.log(S / K) + (r - stdev ** 2 / 2) * T) / (stdev * np.sqrt(T))
```

The difference between d_1 and d_2 is that in d_1, we add the variance divided by 2, denoted as (*r + stdev **2/2*), while in d_2, we must subtract it, denoted as (*r - stdev **2/2*).

To apply the Black Scholes formula, we won't need the PPF distribution we used previously when we forecasted the future stock prices. Instead, we will need the cumulative normal distribution. The reason this is needed is that it shows us how the data accumulates over time. Its output can never be below zero or above one.

We will use the SciPy library and the `cdf()` function, which will take a value from the data as an argument and show us what portion of the data lies below that value.

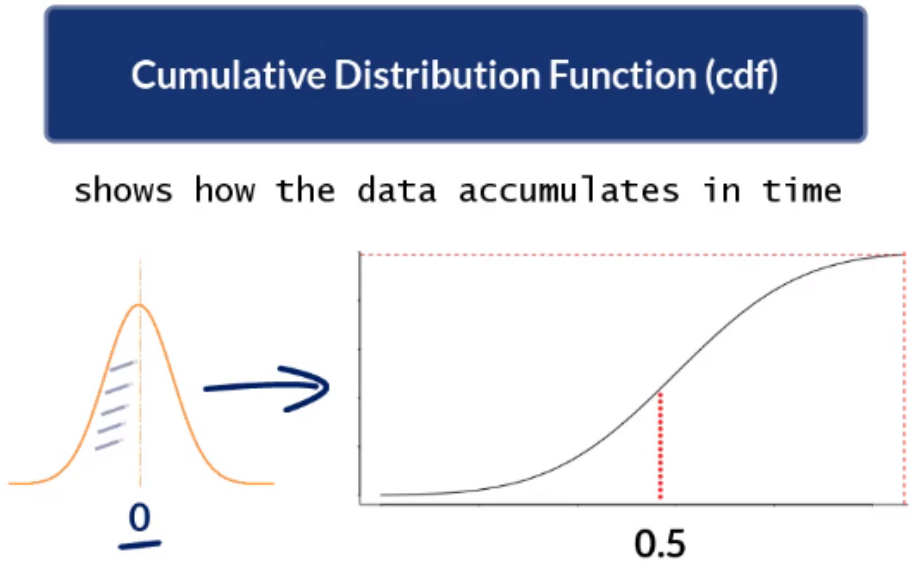
For instance, an argument of `0` will lead to an output of `0.5`, as shown in the following code:

```
In[3]:
norm.cdf(0)
```

The output is as follows:

```
Out[3]:
0.5
```

Since zero is the mean of the standard normal distribution, half the data lies below this value:



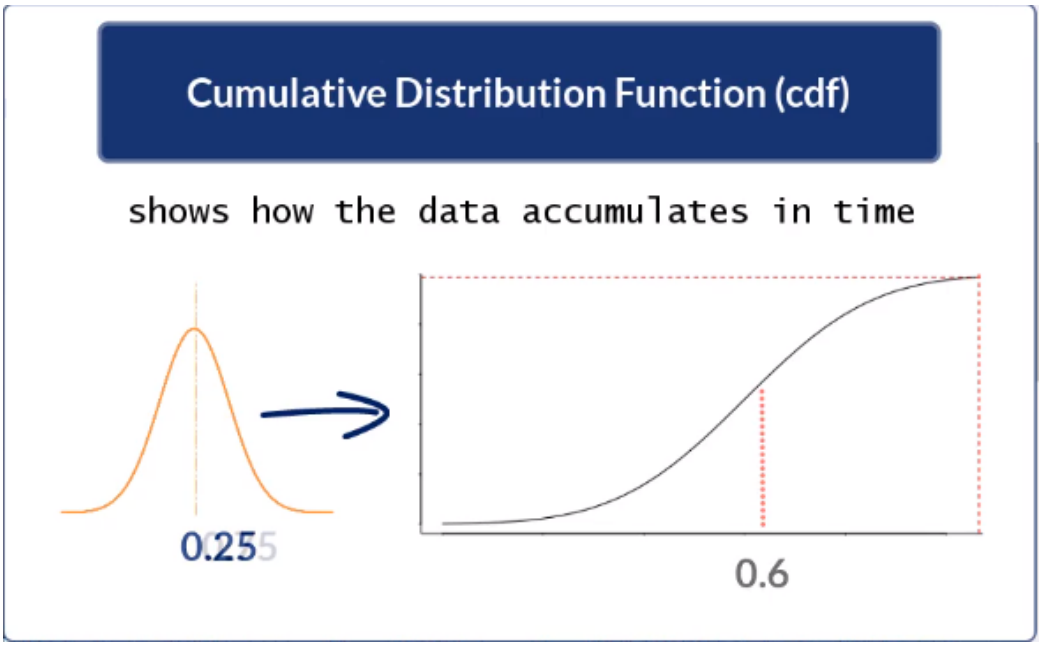
Now, let's give an argument of `0.25`:

```
In[4]:
norm.cdf(0.25)
```

The output is as follows:

```
Out[4]:
0.5987063256829237
```

This can be represented as follows:



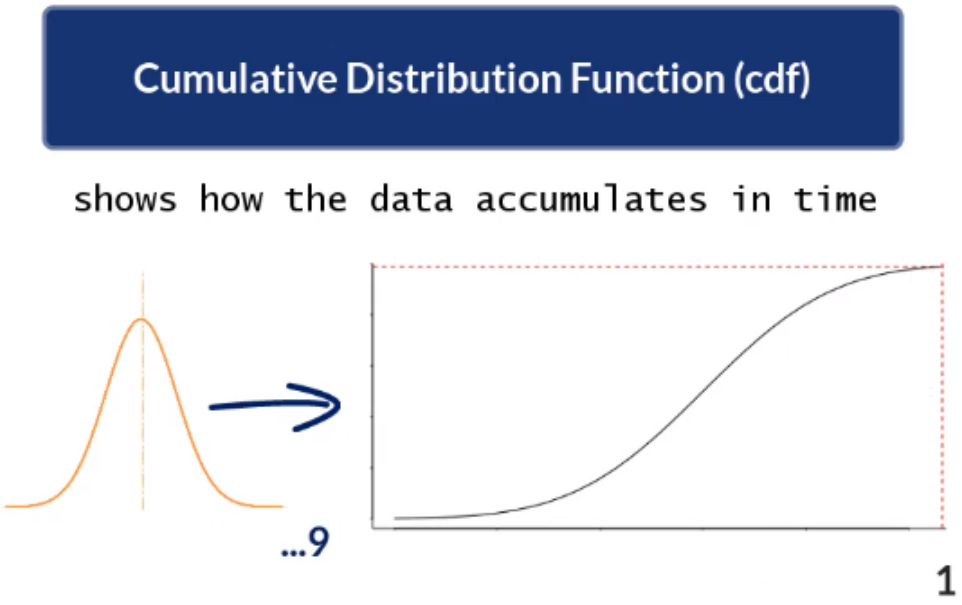
If we give a big argument, such as `9`, we expect to find the largest data point in our set:

```
In[6]:
norm.cdf(9)
```

The output is as follows:

```
Out[6]:
1.0
```

The following diagram gives us a better representation of this:



We can now introduce the Black Scholes function. It will have the same parameters as d_1 and d_2:

$$C = SN(d_1) - Ke^{-rt}N(d_2)$$

The code is as follows:

```
In[7]:
def BSM(S, K, r, stdev, T):
        return (S * norm.cdf(d1(S, K, r, stdev, T))) - (K * np.exp(-r * T) *
norm.cdf(d2(S, K, r, stdev, T)))
```

We have now defined all the necessary functions. Let's apply them to the Procter and Gamble dataset, which consists of the stock prices of the Procter and Gamble company. We will first read the dataset using the pandas `read_csv` function, as shown here:

```
In[8]:
data = pd.read_csv('PG_2007_2017.csv', index_col = 'Date')
```

We will then use the `iloc` operator with the argument as `-1` to get the last record. We will store the last record in the variable `S`, as shown here:

```
In[9]:
S = data.iloc[-1]
S
```

```
Out[9]:
PG    88.118629
Name: 2017-04-10, dtype: float64
```

Another argument we can extract from the data is the standard deviation. In our case, we will use an approximation of the standard deviation of the logarithmic returns of this stock, as shown here:

```
In[10]:
log_returns = np.log(1 + data.pct_change())
```

Then, we will find the standard deviation, as shown here:

```
In[11]:
stdev = log_returns.std() * 250 ** 0.5
stdev
```

The output is as follows:

```
Out[11]:
PG    0.176109
dtype: float64
```

We can now calculate the price of the call option. We will stick to a risk-free interest rate (`r`) of 2.5%, corresponding to the yield of a 10 year government bond. Let's assume that the strike price (`K`) equals $110 and that the time (`T`) is 1 year. Let's define these variables, as follows:

```
In[12]:
r = 0.025
K = 110.0
T = 1
```

At this stage, we have the values for all five parameters we are interested in. If we use them in the three functions we created, we will be able to determine the value for `d1`, `d2`, and the `BSM` function, which are the components of the Black Scholes formula, as shown here:

```
In[13]:
d1(S, K, r, stdev, T)
```

The output is as follows:

```
Out[13]:
PG   -1.029416
```

```
dtype: float64
```

The same can be applied for `d2`, as shown here:

```
In[14]:
d2(S, K, r, stdev, T)
```

The output is as follows:

```
In[14]:
PG   -1.205525
dtype: float64
```

The same can be applied for `BSM`, as shown here:

```
In[15]:
BSM(S, K, r, stdev, T)

Out[15]:
PG    1.132067
Name: 2017-04-10, dtype: float64
```

The call price option from the preceding output is close to $1 and 13 cents. We were able to successfully price the call option. Is it possible to have a call option price that is much lower than the actual stock price? The value of the option depends on multiple parameters, such as the strike price, the time when we are exercising the option, the maturity of the option, and the market volatility. It is not directly proportional to the price of the security.

If you rerun the same code with different values for the time to maturity period, the standard deviation, or the strike price, you will obtain a different option price. In the next section, we will see how we can calculate the price of a stock option in a more sophisticated way.

Using Monte Carlo in conjunction with Euler Discretization in Python

In this section, we will be looking at some more advanced features of finance and mathematics. We will use the Euler Discretization technique to calculate the call option. This is a more sophisticated way of calculating the call option than the techniques we discussed in the previous section. To compare the two approaches, we will use the same dataset we used in the previous section, which is the Procter and Gamble dataset for the time period from January 1, 2007 until March 21, 2017.

To begin with, let's import all the necessary libraries, as shown here:

```
In[1]:
import numpy as np
import pandas as pd
from pandas_datareader import data as web
from scipy.stats import norm
import matplotlib.pyplot as plt
%matplotlib inline
```

The next step is to read the dataset using the pandas `read_csv()` method, as shown here:

```
In[2]:
data = pd.read_csv('PG_2007_2017.csv', index_col = 'Date')
```

In the next step, we calculate the log returns, as shown here:

```
In[3]:
log_returns = np.log(1 + data.pct_change())
```

In Euler Discretization, the methods and formula we will apply to compute the call option price are different. We want to run a huge number of experiments to make sure that the price we pick is the most accurate one.

Monte Carlo simulation can provide us with thousands of possible call option prices. We can then average the pay off. The trick lies in the formula we will use to calculate the future prices, which is as follows:

$$S_t = S_{t-1} . e^{((r - \frac{1}{2} . stdev^2) . \delta_t + stdev . \sqrt{\delta_t} . Z_t)}$$

This is another version of a Brownian motion. The formula and the approach employed are what is known as Euler Discretization. The following are the parameters of the preceding equation:

- S_t = the stock price for day *t*
- S_{t-1} = the stock price observed on the previous day, *t-1*

Let's go through the steps that will allow us to assign values in this long formula.

The first thing that we will do is assign a risk-free interest rate (`r`). The initialization is shown here:

```
r = 0.025
```

Here, we assign the value of the risk-free interest rate as 2.5%. Next, we can obtain the standard deviation of the log returns and store it in a `stdev` variable, as shown here:

```
In[5]:
stdev = log_returns.std() * 250 ** 0.5
stdev
```

The output is as follows:

```
Out[5]:
PG    0.176109
dtype: float64
```

We can verify that the `stdev` variable is a series using the following code:

```
In[6]:
type(stdev)
```

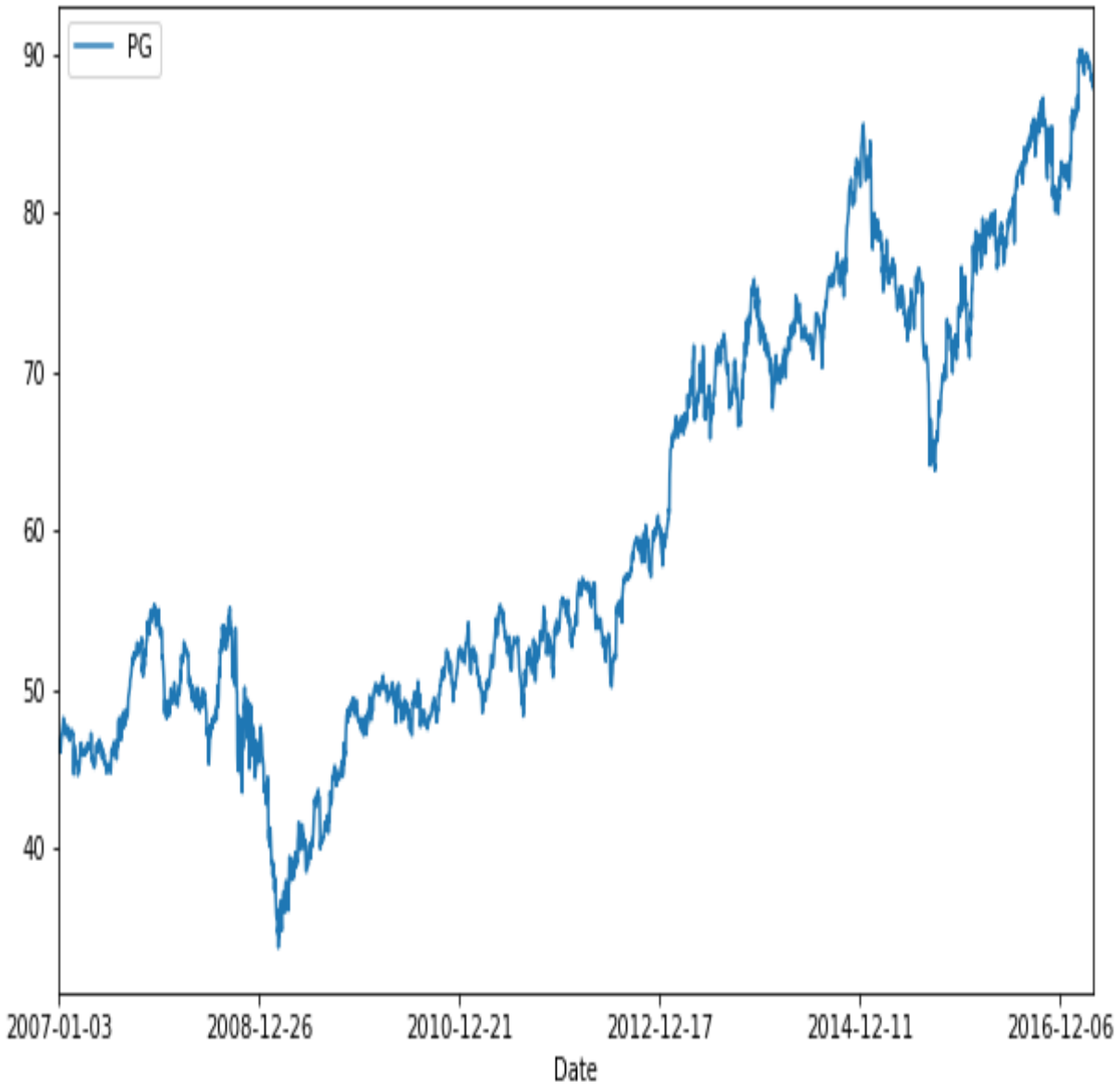
The output is as follows:

```
Out[6]:
pandas.core.series.Series
```

Let's plot and check the data of the Proctor and Gamble dataset, as shown here:

```
In[6]:
data.plot(figsize=(10, 6));
```

The output is as follows:



We intend to use the `stdev` variable in an array operation afterwards, so it is necessary to convert the `stdev` variable, which is a series, into an array, by using values, as shown here:

```
In[7]:
stdev = stdev.values
stdev
```

The output is as follows:

```
Out[7]:
array([ 0.17610875])
```

The preceding output basically tells us that `stdev` is now a NumPy array.

We will consider the time (`T`) to be 1 year, since we are forecasting the prices for 1 year ahead. The number of time intervals (`t_intervals`) must correspond to the number of trading days in a year, which is `250`. The initialization is shown in the following code:

```
In[8]:
T = 1.0
t_intervals = 250
delta_t = T / t_intervals

iterations = 10000
```

From the preceding code, we have created two more variables: `delta_t`, which is the fixed time interval, and the 10,000 simulations or iterations for simulating `z`, which is the random component.

The code is shown here:

```
In[9]:
Z = np.random.standard_normal((t_intervals + 1, iterations))
```

The random component `z` will be a matrix with random components drawn from a standard normal distribution, which is a normal distribution with a mean of 0 and a standard deviation of 1.

The dimension of the matrix will be defined by the number of time intervals augmented by 1 (`t_intervals` + 1) and the number of iterations (iterations).

The code is as follows:

```
In[10]:
S = np.zeros_like(Z)
```

In this step, we can create an empty array, `S`, of the same dimensions as the random component `Z`. For this, we use the `np.zeros_like()` method, as shown in the preceding code.

Then, we will use the `iloc` operator with the argument as `-1` to get the last record. We will store the last record in the variable `S0`, as shown in the preceding code:

```
In[11]:
S0 = data.iloc[-1]
S[0] = S0
```

As well as changing the formula, which differs from the one we used for calculating the future stock prices, the remaining steps are almost identical to the Monte Carlo technique we looked at in the previous chapter.

We loop from `1` to the number of intervals (`t_intervals + 1`) and we create a whole stock price matrix with the dimensions of (`t_intervals + 1`) and the number of iterations, as shown here:

```
In[12]:
for t in range(1, t_intervals + 1):
    S[t] = S[t-1] * np.exp((r - 0.5 * stdev ** 2) * delta_t + stdev * delta_t **
0.5 * Z[t])
```

We can see the value inside `S`, as follows:

```
In[13]:
S
```

The output is as follows:

```
Out[13]:
array([[  88.118629  ,   88.118629  ,   88.118629  , ...,   88.118629  ,
          88.118629  ,   88.118629  ],
       [  88.81636639,   90.61639177,   89.03898033, ...,   87.30375254,
          87.5583302 ,   88.80166136],
       [  86.90009657,   91.62555653,   89.63268689, ...,   88.33679858,
          87.62299858,   91.35123942],
       ...,
       [  84.3786526 ,  107.72224183,   80.82567392, ...,  125.62838195,
```

```
          89.32732597,  105.18086595],
       [  84.3872724 ,  108.28857142,   81.12333622, ...,  125.84132277,
          90.90533155,  105.51622291],
       [  83.67055401,  107.96691607,   81.7082641 , ...,  125.94172348,
          91.80990436,  104.04308705]])
```

We can check the dimension by using the `shape` property, as shown here:

```
In[14]:
S.shape
```

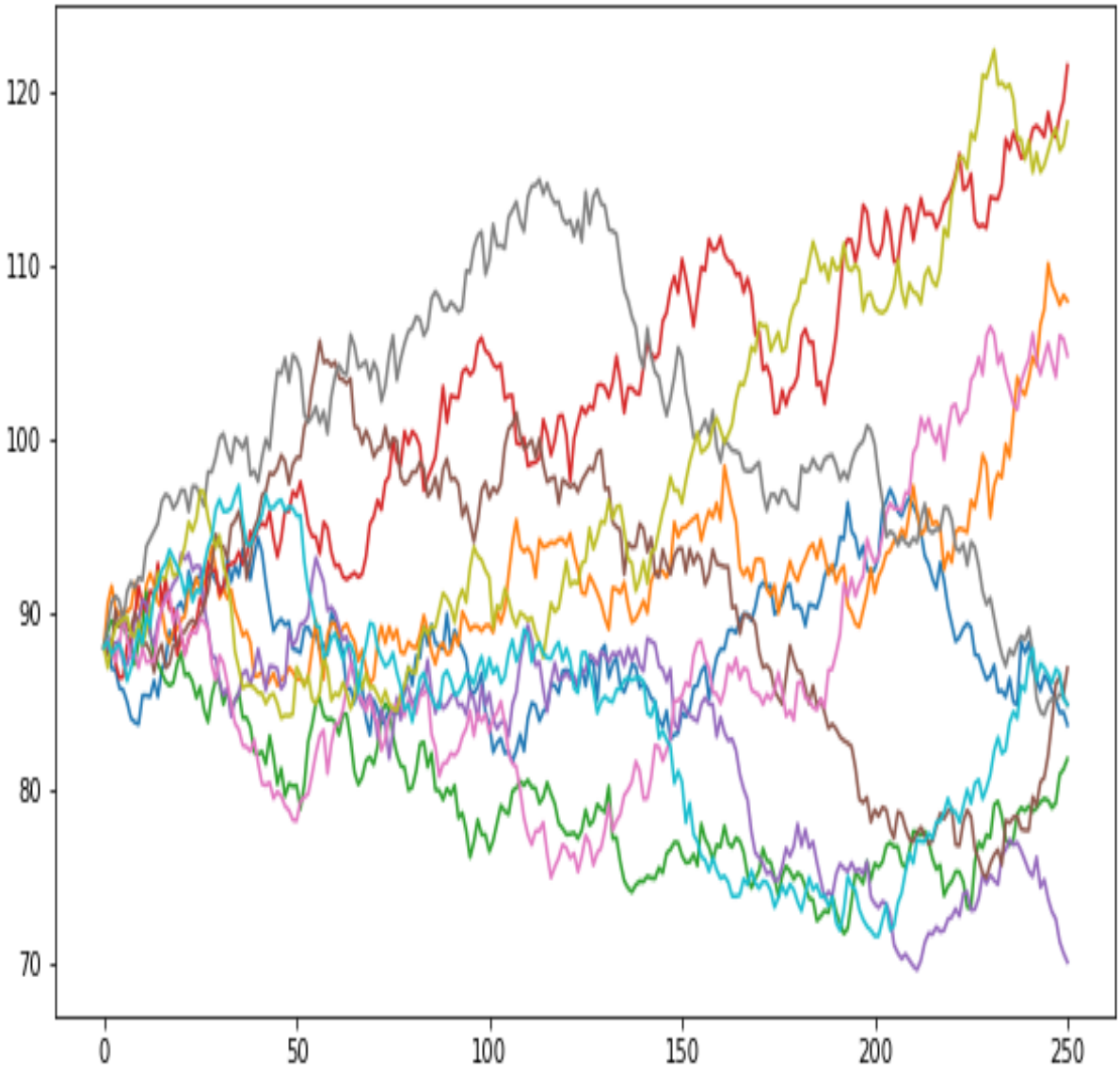
The output is as follows:

```
Out[14]:
(251L, 10000L)
```

To plot only 10 simulations, we can use matplotlib. The code is shown here:

```
In[15]:
plt.figure(figsize=(10, 6))
plt.plot(S[:, :10]);
```

The output is as follows:



Our simulation is complete and now we will go ahead and compute the call option price.

Let's see what the payoff is like for a call option. At a certain point in time, we will exercise our right to buy the option if the stock price minus the strike price is greater than zero. We will not exercise our right to buy if the difference is a negative number.

So, the value of the option depends on the chance of the difference between the stock price (S) and the strike price (K) being positive and, in particular,

how positive it is expected to be.

To work this out, we can use a NumPy method called `maximum()`, which will create an array that contains either zeros or the numbers equal to the differences. We call `p`, which represents the payoff. The code is as follows:

```
In[16]:
p = np.maximum(S[-1] - 110, 0)
p
```

The output is as follows:

```
Out[16]:
array([ 0.,  0.,  0., ...,  0.,  0.,  0.])
```

We can also check the shape of `p`, as shown here:

```
In[17]:
p.shape
```

The dimensions are as follows:

```
Out[17]:
(10000L,)
```

The formula to discount the average of the payoff *p* is shown here:

$$C = \frac{e^{(-rT)} \cdot \sum p_i}{iterations}$$

This corresponds to the exponential of the *r* and *T* multiplied by the sum of the computed payoffs, divided by the number of iterations we choose, which is 10,000.

The preceding equation can be written in Python, as shown here:

```
In[18]:
C = np.exp(-r * T) * np.sum(p) / iterations
C
```

The output is the price of the call option:

```
Out[18]:
1.159769289926591
```

It is important to compare whether this number differs substantially from the one we obtained with the Black Scholes formula in the previous section. From the Black Scholes formula, we got a call option value of 1.13. This is very close to the value of 1.15, which we got by using Euler Discretization. This is proof that the choice of computation method can lead to insignificant but not unimportant differences in the outcome.

We are now armed with enough pythonic skills to conduct these kinds of studies on our own.

Summary

In this chapter, we have looked at various concepts, such as derivative contracts, the Black Scholes formula, and Euler Discretization. We used these techniques to calculate the call option price, which in turn can be used in derivative contracts. We also implemented this practically by using Python. We then looked at the difference between the call option price that's calculated using the different techniques.

In the next chapter, we are going to introduce deep learning techniques, which can be used to solve problems such as stock prediction, sales prediction, and more. We will look at different frameworks, such as TensorFlow and Keras, which help us to build a basic architecture called a neural network. We will be looking at a famous neural network architecture called a recurrent neural network, which is a very important technique that works well on sequences of data.